CTX

문맥을 프로그램으로 만드는 법

prompt·tool schema·history·memory·compact·cache가 하나의 실행 입력을 구성하는 방식

> **모델 품질은 호출 함수보다 모델이 실제로 보게 되는 문맥의 구조·순서·보존 규칙에서 결정된다.**

대상
v2.1.88 / 2ca5ddabfed5

관점
context compiler · cache · history

제3자 복원 소스에 대한 교육용 분석이다. 누락된 모듈과 복원 한계를 구현 사실로 보완하지 않는다.

context는 문자열이 아니라 컴파일 결과다

최종 API 입력은 사용자 prompt 하나가 아니다. system policy, 환경, 도구 schema, 대화 이력, attachment, memory, compact projection을 규칙에 따라 조립한 값이다.

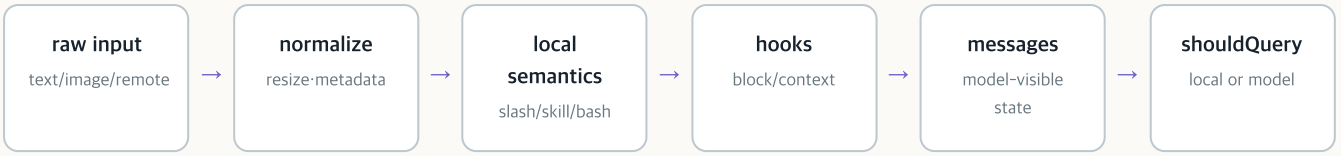
개념 모델 — 실제 타입 복사 아님

```
ContextProgram = compile({
  policy, environment, tools,
  conversation, attachments, memory,
  modelCapabilities, tokenBudget
})
```

따라서 prompt engineering만 보면 핵심을 놓친다. 무엇을 포함하고 제외하는지, 어떤 순서가 cache key를 만드는지, 어떤 메시지가 UI에는 보이지만 API에는 제거되는지가 더 결정적이다.

입력은 곧바로 모델에 가지 않는다

processUserInput은 string과 content block을 정상화하고 slash command, image, IDE selection, attachment, hook을 구분한 뒤 shouldQuery와 추가 허용 도구를 함께 반환한다.



여기서 중요한 설계는 “입력 텍스트”와 “turn 실행 여부”를 분리한 것이다. `/help` 같은 local command는 conversation에 흔적을 남길 수 있지만 model call은 만들지 않는다. 반대로 hook의 additional context는 사용자 입력이 아니어도 model-visible message가 된다.

코드 좌표 src/utils/processUserInput/processUserInput.ts · src/utils/processUserInput/processUserInput.ts

같은 문자열도 출처에 따라 권한이 다르다

user text, system policy, hook output, skill content, tool result, memory는 모두 model context가 될 수 있지만 신뢰 수준과 수명은 다르다.

출처	모델에 주는 역할	필요한 제약
system prompt	행동 정책·도구 사용법	cache 안정성, build별 차이
user prompt	현재 목표	명령/붙여넣기/원격 입력 구분
repository memory	프로젝트 규칙과 맥락	경로·중복·freshness
hook context	조직/로컬 자동화 결과	길이 제한, block 의미
tool result	외부 세계 관찰	tool_use id, size, error 표현
synthetic/meta message	runtime 제어 힌트	UI visibility와 API visibility 분리

보안 연결

context에 들어갔다고 authority가 생긴 것은 아니다. untrusted repository text나 tool output은 모델 행동에 영향을 줄 수 있으므로, 실제 effect는 별도의 permission boundary에서 다시 매개해야 한다.

system prompt는 정적 템플릿이 아니다

모델, 활성 도구, 추가 작업 디렉터리, MCP, memory, 사용자 지정 prompt에 따라 섹션이 선택되고 결합된다.

01	**기본 행동 계약**	task 수행, 도구 사용, 응답 방식
02	**session guidance**	현재 모드·환경·사용자 설정
03	**memory**	CLAUDE.md/MEMORY 계열 지식
04	**tool-specific guidance**	활성 도구의 prompt와 capability
05	**environment**	cwd, platform, shell, repository 상태
06	**append/custom prompt**	SDK caller의 교체 또는 추가 정책

custom system prompt는 기본 prompt를 보완하는 것이 아니라 대체할 수 있다. 이 차이는 systemContext 포함 여부까지 바꾸므로, “문자열 concatenate”가 아니라 구성 정책이다.

코드 좌표 src/constants/prompts.ts · src/utils/queryContext.ts

도구 목록은 실행 registry이면서 모델의 프로그래밍 언어다

모델은 tool name, description, JSON schema를 보고 행동 표현을 생성한다. 도구 순서와 schema 존재 여부가 모델 출력과 prompt cache를 동시에 바꾼다.

의미

capability vocabulary

Read/Edit/Bash 같은 동사가 모델의 행동 언어가 된다.

형식

JSON schema

자유 텍스트 의도를 runtime이 검증할 구조로 제한한다.

비용

token footprint

도구 수와 설명 길이가 매 요청 context 비용이 된다.

캐시

stable prefix

built-in과 MCP 정렬·경계가 cache reuse를 좌우한다.

코드 좌표 src/tools.ts · src/tools.ts · src/services/api/claude.ts

활성 tool pool은 permission과 런타임 상태의 함수다

registry에 존재한다고 모델에게 항상 노출되는 것은 아니다. mode, environment, deny rule, MCP connection, ToolSearch discovery가 pool을 바꾼다.

```
VisibleTools = filter(mode, env, denyRules, MCP, discovery, modelSupport)
```

여기서 두 종류의 차단을 구분한다. **노출 전 filtering**은 모델이 capability를 계획하지 못하게 하고, **dispatch-time permission**은 실제 입력을 보고 실행을 막는다. 전자만으로 complete mediation이 되지 않으며 후자는 매 호출마다 필요하다.

코드 좌표 src/tools.ts · src/services/tools/toolExecution.ts

conversation은 문자열 배열이 아니다

assistant, user, attachment, progress, system, tombstone, stream_event는 API·UI·transcript에서 서로 다른 운명을 가진다.

종류	핵심 의미	projection 차이
assistant	model output와 tool_use	SDK event, transcript, 다음 API 모두 관여
user/tool_result	사용자 목표 또는 effect 관찰	tool_result pairing이 protocol 불변조건
attachment	파일 변화·hook·queue·memory 등 부가 문맥	일부는 API normalize 시 user content로 변환
progress	장기 effect의 중간 관찰	UI에는 중요하지만 최종 reasoning에는 축약 가능
tombstone	실패 attempt의 기존 projection 제거	새 데이터가 아니라 삭제 제어 신호
system boundary	compact/snip/retry 같은 runtime 사건	resume와 SDK에서 선택적으로 노출

복원 한계

`src/types/message.ts`는 이 복원본에서 누락됐다. 이 분류는 import/use site에서 확인한 동작 대수이며 원문 타입 선언을 복원한 것이 아니다.

한 상태에서 여러 표현을 만든다

내부 Message를 API parameter, SDKMessage, terminal UI row, JSONL transcript로 그대로 복사하지 않는다. 소비자 계약에 맞는 projection이 필요하다.

하나의 사건, 여러 projection

```
internal message
  ├── normalizeMessagesForAPI() → provider protocol
  ├── normalizeMessage()        → SDK event
  ├── Messages/Tool UI          → terminal projection
  └── recordTranscript()        → durable JSONL
```

이 구조의 위험은 projection drift다. 한 새 message variant를 추가했는데 API normalizer만 수정하면 UI나 resume가 침묵할 수 있다. union exhaustiveness와 contract test가 필요한 이유다.

코드 좌표 src/utils/messages.ts · src/QueryEngine.ts

읽기는 정보 획득이자 쓰기 권한의 전제다

FileReadTool은 결과를 보여 주는 데서 끝나지 않고 file path, content, timestamp, partial-view 여부를 readFileState에 남긴다. Edit는 이 state를 optimistic lock처럼 사용한다.

Read-before-write는 prompt 지시가 아니다	Edit.validateInput과 call이 runtime에서 강제한다. 모델이 "읽었다"고 말하는 것과 readFileState에 full read가 존재하는 것은 다르다.

이 연결 때문에 context와 effect가 분리되어도 완전히 독립적이지 않다. 관찰의 freshness가 후속 효과의 admissibility를 결정한다. partial read는 내용 일부만 알기 때문에 전체 파일 edit의 전제로 인정되지 않는다.

코드 좌표 src/tools/FileReadTool/FileReadTool.ts · src/tools/FileEditTool/FileEditTool.ts

tool output은 무한히 context에 남을 수 없다

검색·shell·MCP 결과가 커질수록 다음 model call의 비용과 한계가 급격히 커진다. tool별 maxResultSizeChars와 context replacement가 이를 제어한다.

전략	무엇을 보존	잃을 수 있는 것
tool 자체 제한	가장 관련 있는 줄/페이지	전체 원문
disk persistence + preview	추후 Read 가능한 경로와 요약	즉시 context의 세부
tool result budget	최근/필수 결과	오래된 대용량 출력
microcompact/snip	tool id와 경계 정보	원래 result 내용
semantic summary	과업 진행에 필요한 의미	정확한 재현성

설계 trade-off

context 절약은 단순 truncation이 아니다. 이후 모델이 어떤 증거를 다시 읽을 수 있는지와 resume가 동일 projection을 재구성할 수 있는지를 함께 설계해야 한다.

압축은 하나의 기능이 아니라 계층화된 정책이다

query loop는 tool-result budget, snip, microcompact, context collapse, autocompact를 서로 다른 시점과 의미로 적용한다.

01	result budget	개별 대용량 결과를 교체하고 필요하면 record를 저장
02	snip	특정 history 구간을 제거하고 boundary를 남김
03	microcompact	tool 결과를 id 중심으로 줄임
04	context collapse	full history 위에 read-time projection/commit log를 적용
05	autocompact	summary message와 attachment로 새 context window 생성

순서에는 의미가 있다. 예를 들어 snip에서 확보한 token은 autocompact threshold 판단에 반영되어야 하고, collapse가 충분하면 더 손실이 큰 summary를 피할 수 있다.

코드 좌표 src/query.ts

prompt cache는 의미가 아니라 bytes와 순서를 본다

동일한 도구 집합도 순서가 달라지면 server-side prefix cache가 깨질 수 있다. 이 때문에 built-in tool을 안정된 prefix로 정렬하고 MCP tool을 뒤에 둔다.

cache-aware request layout

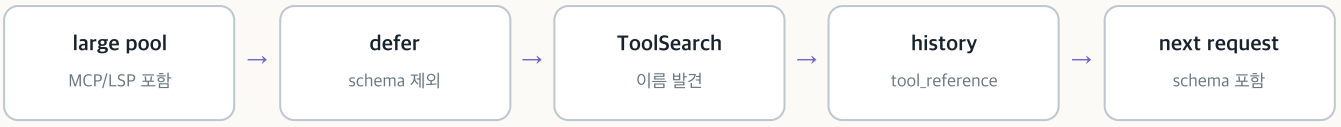
```
stable prefix:
[system sections][built-in tool schemas][cache breakpoint]
dynamic suffix:
[MCP tools][advisor/feature-specific additions][messages]
```

이 설계는 데이터 구조 선택을 바꾼다. 관찰용 backfill을 원본 assistant message에 직접 쓰지 않고 clone하는 이유도 다음 request의 byte identity를 보존하기 위해서다.

코드 좌표 src/tools.ts · src/query.ts · src/services/api/claude.ts

도구 발견은 context footprint를 줄이는 paging이다

deferred tool은 첫 요청에 전체 schema를 싣지 않고 ToolSearch 결과가 history에 나타난 뒤에만 포함된다.



이 최적화는 새 실패 모드를 만든다. schema가 전달되지 않은 상태에서 모델이 typed parameter를 문자열로 만들 수 있고, dispatch parser가 이를 거부한다. 그래서 error가 단순 Zod 메시지가 아니라 “먼저 tool을 load하라”는 복구 protocol을 포함한다.

코드 좌표 src/services/api/claude.ts · src/services/tools/toolExecution.ts

원본과 관찰용 표현을 분리한다

같은 tool input이라도 API 재전송, hook 관찰, SDK stream, transcript에서 필요한 필드가 다르다.

값	허용되는 변환	이유
API-bound assistant message	가급적 원본 유지	prompt cache와 signed thinking block 보존
observer/tool input clone	legacy·derived 필드 backfill	hook/SDK 호환성
permission-updated input	정책이 승인한 새 값 사용	사용자/자동화 결정 반영
tool.call input	schema parse 결과와 명시적 update만	관찰 편의 mutation이 실제 effect를 바꾸지 않게 함

> **배경지식**
> 이는 immutability 취향이 아니라 **representation ownership** 문제다. 어느 consumer가 어떤 모양의 값을 소유하는지 불명확하면 cache, transcript hash, hook semantics가 함께 깨진다.

코드 좌표 src/query.ts · src/services/tools/toolExecution.ts

context 오류는 늦게 나타난다

문맥 구성 버그는 즉시 exception보다 다음 request, resume, provider 전환에서 드러나는 경우가 많다.

오류	지연된 증상	설계 대응
orphan tool_result	다음 API call 400	pair repair 또는 synthetic result
stale tool-search field	다른 모델/provider에서 400	model-specific post-processing
cache prefix drift	비용·TTFT 악화	stable ordering과 cache break detection
compact metadata 누락	resume에서 pre-compact history 부활	boundary/relink/projection 기록
중복 memory attachment	context 팽창	session-level loaded path set
UI-only message 유입	모델이 presentation text를 사실로 해석	normalize boundary에서 제거

연결할 배경지식: compiler와 materialized view

context pipeline은 소스 정보를 읽어 target protocol을 만드는 compiler와 닮았다. transcript/compact projection은 materialized view와 닮았다.

compiler

normalization passes

raw input → typed message → provider request. 각 pass의 입력/출력 계약이 필요하다.

database

projection & compaction

원본 history와 읽기 최적화 view를 구분한다.

distributed systems

cache key & replay

같은 의미보다 deterministic serialization이 중요할 수 있다.

security

provenance

context의 출처와 effect authority를 분리한다.

코드 좌표 Event Sourcing — projection 관점 · TypeScript narrowing

context compiler를 명시적 단계로 만들기

현재 로직을 단순히 한 파일로 모으면 dependency cycle과 feature 결합이 커진다. 대신 immutable intermediate representation과 cache contract를 먼저 정의한다.

제안하는 분해 방향

```
RawInput
 → InputIR { user, localCommand, attachments, provenance }
 → ConversationIR { messages, toolRefs, boundaries }
 → RequestIR { system, tools, messages, cacheSegments }
 → ProviderRequest
```

- **각 단계는 pure projection을 기본**으로 하고 effect는 loader/adapter에 둔다.
- **cacheSegments를 값으로 표현**해 순서와 dynamic suffix를 검증한다.
- **provenance를 유지**해 UI visibility, model visibility, authority를 혼동하지 않는다.
- **compact/snip을 rewrite policy**로 만들고 replay equivalence test를 둔다.

> **우선 보존할 oracle**
> 동일 fixture에 대해 provider request JSON, cache segment hash, tool_use/result pairing, resume 후 message projection이 같아야 한다.

한 Edit turn에서 context가 하는 일

파일 수정 기능을 실제로 성립시키는 문맥은 prompt보다 넓다.

1. system prompt가 Read-before-Edit 사용법과 도구 vocabulary를 제공한다.
2. 사용자 prompt와 repository memory가 대상 변경의 의미를 제공한다.
3. Read 결과가 내용과 timestamp를 message/readFileState 양쪽에 남긴다.
4. Edit schema가 old_string/new_string/file_path라는 행동 언어를 제한한다.
5. Edit result가 patch와 성공 정보를 다음 reasoning에 되돌린다.
6. 테스트 Bash result가 변경이 목표를 만족하는지 새 context가 된다.
7. transcript와 compact policy가 다음 turn/resume에서 필요한 부분을 보존한다.

학습 포인트	“prompt가 좋아서 편집했다”가 아니라, 관찰·행동 schema·runtime guard·결과 환류가 함께 모델의 작업 능력을 만든다.

좋은 context 설계의 판단 기준

더 많은 정보를 넣는 것이 목표가 아니다. 올바른 정보가 올바른 표현과 수명으로 들어가고, 비용·cache·resume 조건 아래에서도 의미가 유지되어야 한다.

질문	좋은 답의 형태
무엇이 authoritative한가?	filesystem/read state/transcript와 UI projection을 구분
무엇이 model-visible한가?	normalize 후 provider request로 확인
무엇이 cache key인가?	stable prefix와 dynamic suffix를 명시
무엇을 버릴 수 있는가?	재읽기·요약·boundary로 복구 가능한 정보만
resume는 같은 의미를 재생하는가?	compact/snip/tool pair까지 equivalence test